Table

A table cell shows a list of records as rows and columns that people can search, sort, group, pick from and act on; on a narrow screen each row becomes a card.

In notation	In the / menu	Holds	Raises
`["table", …]`	**Table**	no other cells	`click`, `dblclick`, `pick`, `change`

When to use it

Use a table when there are several records with the same fields: invoices, people, orders, a price list. The rows can be typed into the table itself, saved by buttons in a collection, or computed by a formula.

For a short list of single items to tick off, use a list. For one number, use a stat; to see the numbers in a table as shapes, hand the same expression to a chart.

Try it

Invoices Runs in this page. Records it saves live in memory and are gone when you leave.

Click *total* in the header to sort by it; click again to reverse, and once more to go back to the order the rows came in. Type in the search box to keep only the rows where a column contains what you typed. Fold a group by its heading. Tick two invoices and **Picked** adds them up. **Mark paid** asks first, then changes the invoice's record, and the row moves to the *Paid* group.

The rows come from the collection `invoices`, which the example brings with it; the table shows the fields you list in `columns`. Every table's value is its list of rows, so formulas can count and add them:

Expression

```
(len board)
⇒ 4
(sum-by (get it "total") board)
⇒ 5800
(map (get it "client") (where board "status" "Late"))
⇒ (list "Harbor Bakery")
(update! "invoices" "r1" {status "Paid"})
(map (get it "client") (where board "status" "Paid"))
⇒ (list "Ada Lovelace" "Northwind Studio")
```

Adding and designing one

1. Select a cell and press `/`, then choose **Table**.
2. A new table holds three typed rows (*Tea*, *Cake*, *Juice*) to show how it works. Double-click a cell in the table to change it; **Add a row** adds a line and the **+** at the end of the header adds a column.
3. Name it in **Name**, at the top of **Cell details**, if other cells will read what is picked.

roster

Table

c1

Rows of records people can search, sort and pick. Table in the docs

Name

roster

Name it to use it in formulas.

Content

DATA

Rows come from

Typed here | Saved records | Formula

3 rows. Double-click a cell in the table to change it.

+ Add a row + Add a column

COLUMNS

- [x] name
- [x] team
- [x] nights

Show every column, as it comes

Numbers in every column

Automatic

ROWS

Group by

No grouping

Rows with the same value sit under one heading.

Picking rows

Off | One | Many

Let people pick rows for other cells to use.

ROW BUTTONS

No buttons yet. They show at the end of every row.

ADD A BUTTON

Change a field
e.g. mark an order Done

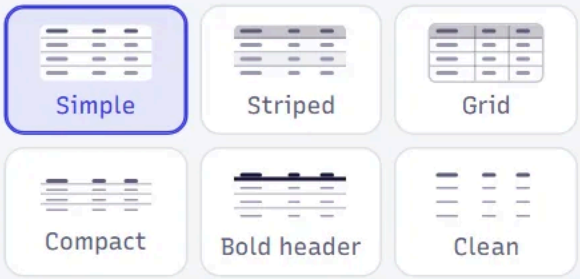
Delete the record
Asks first

Copy into a cell
Fill an input from the row

Custom
Write the action yourself

LOOK

Quick looks



Lines — Between rows

Header row

Plain | Filled | Strong | None

Row spacing

Compact | Normal | Roomy

Stripes — Search box

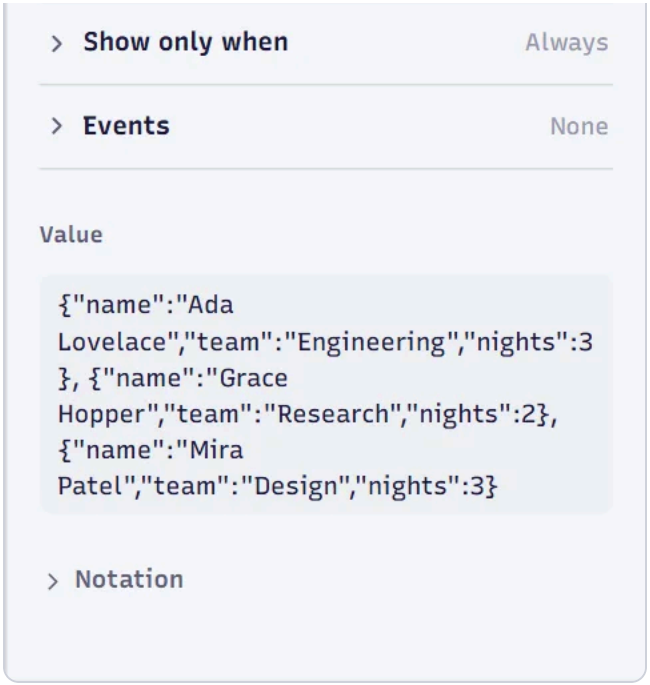
Title

The team

Text — Default

Spacing and size

Box



The table's designer: where the rows come from, the columns, grouping and picking, buttons on every row, and the look.

The **Content** section has five parts.

Data

Rows come from chooses the source:

- **Typed here**: the rows are the table's own value. Double-click a cell to change it in Edit; Enter keeps the change and goes down, Tab goes to the next cell, Esc drops it (shortcuts). Typed text becomes a number when the other values in its column are numbers (or there are none yet), and `true` or `false` in a column of yes and no. Text such as `007`, with leading zeros, stays text when it is the first value. The × at a row's end deletes the row.
- **Saved records**: pick a **Collection**, or **New collection…**. The rows are the records saved by buttons in any of your documents, kept up to date as they change. See Collections.
- **Formula**: any expression giving a list of records, such as `(where (rows "orders") "paid" true)`.

Columns

Every field of the rows is a column, in the order the fields first appear; `id` is left out. Tick the ones to show, move them with the arrows, and press a column's name to open its settings:

- **Heading**, and for typed rows the **Field name** (renaming changes every row).
- **Show as**: **Text**, **Badge**, **Progress**, **Tick**, **Link** or **Stars**. A badge gets one colour per value in **Badge colours**; an unmapped value is grey. Progress draws a bar for a number from 0 to 1 (or 0 to 100). A tick is a check mark for anything but empty, `false`, `no`, `0` or `n`. A link opens a web address, a `mailto:` address, an email address or a bare domain such as `example.test`. Stars round a number to 0–5.
- **Numbers as**: the format; **Align**, **Width** (px), **Bold**, **Wrap long text**, **Colour**.
- **Total**: **Sum**, **Average**, **Count**, **Smallest** or **Largest**, shown under the table and under each group. Blanks and text are skipped; count counts the filled values.

Numbers in every column sets one format for all numbers that have none of their own. Dragging the edge of a header also sets a column's width.

Rows

- **Group by** a field: rows with the same value sit under one heading with their count, and the heading folds. **Collapse all** and **Expand all** appear above. Rows with no value are under *(none)*. Columns with a total get a **Subtotal** per group and a **Total**.
- **Picking rows**: **Off**, **One** or **Many**. People pick with a checkbox (or a round pick for one), or by clicking the row in Live. The designer shows *n picked* with **Clear**, and *Other cells read the picked rows with* `(selected name)`.

Row buttons

Buttons shown at the end of every row. **Add a button** offers **Change a field** (set a field of the saved record, such as marking an order *Done*), **Delete the record** (asks *Delete this row?* first), **Copy into a cell** (fills an input from the row) and **Custom**. Each button has a **Label**, an **Icon**, a **Look**, **Ask first** and **What it does**, where `row` is the clicked row's record. **Change a field** and **Delete the record** work on saved records; typed rows are changed by double-clicking them.

Look

Quick looks are **Simple**, **Striped**, **Grid**, **Compact**, **Bold header** and **Clean**. Behind them: **Lines** (between rows, between columns, grid, only around, none), **Header row** (**Plain**, **Filled**, **Strong**, **None**), **Row spacing** (**Compact**, **Normal**, **Roomy**), **Stripes**, **Search box** and a **Title**.

In notation

The rows' expression is the body; typed rows are the `value` prop.

Notation

```
["table", {"name": "team", "label": "The team", "select": "one", "density": "compact",
"stripes": true,
  "columns": ["name", "role",
    {"key": "load", "show": "progress", "label": "Booked"},
    {"key": "remote", "show": "check", "label": "Remote"},
    {"key": "rating", "show": "stars"},
    {"key": "site", "show": "link"}],
  "value": [
    {"name": "Ada Lovelace", "role": "Engineer", "load": 0.8, "remote": true, "rating":
5, "site": "example.test"},
    {"name": "Grace Hopper", "role": "Designer", "load": 0.4, "remote": false,
"rating": 4, "site": "example.test/grace"}]}]
```

Prop	What it is
body or `expr`	An expression giving the rows. Without one, the rows are `value`.
`value`	Typed rows: a list of records.
`label`	A title above the table.
`columns`	Field names, or records `{"key", "label", "format", "width", "align", "show", "colors", "color", "bold", "wrap", "total"}`, in the order to show. `align` is `"start"`, `"center"` or `"end"`; `show` is `"text"`, `"badge"`, `"progress"`, `"check"`, `"link"` or `"stars"`; `colors` maps a badge's values to colour tokens; `total` is `"sum"`, `"avg"`, `"count"`, `"min"` or `"max"`.
`format`	The format of every number in a column that has none.
`group`	A field to group the rows by.
`select`	`"one"` or `"many"` lets people pick rows.
`selected`	The keys of the picked rows.
`actions`	Row buttons: `[{"label", "do", "icon"?, "variant"?, "confirm"?}]`; `do` runs with `row` bound to the record.
`borders`	`"rows"` (default), `"columns"`, `"grid"`, `"outer"` or `"none"`.
`stripes`	`true` shades every other row.
`density`	`"compact"`, `"normal"` (default) or `"roomy"`.
`header`	`"plain"` (default), `"filled"`, `"strong"` or `"none"`.
`search`	`false` hides the search box.

Its value

The table's value is its list of rows, as records: the typed rows, or what the expression gives. Anything in the list that isn't a record is skipped. A record saved in a collection also has an `id` and the time it was saved, `at`, which shows as *5 min ago* when it is a column.

What people pick is kept apart from the value, in `selected`, as a list of keys. A row's key is its `id`, or its place in the rows (from 0) when it has none. Read the picked rows as records with selected:

Notation

```
["table", {"name": "team", "select": "many", "selected": [0, 2],
  "value": [{"name": "Ada", "load": 0.8}, {"name": "Grace", "load": 0.4}, {"name":
"Alan", "load": 0.9}]}]
```

Expression

```
(selected team)
⇒ (list {name "Ada" load 0.8} {name "Alan" load 0.9})
(len (selected team))
⇒ 2
```

Events

Event	Its data
`click`	row: the row clicked, as a record; index: its position in the table's rows (from 0)
`dblclick`	row: the row double-clicked; index: its position
`pick`	rows: the picked rows, as records; value: their keys; was: the keys before
`change`	value: the new value; was: the value before

`pick` fires when the picked rows change, whoever changed them, and binds `rows`, the records now picked. `click` binds `row` and its `index`, where the index is the place in the table's rows before any sort or search. When a table has a `dblclick` handler, a single click waits a moment to be sure it isn't half of a double-click. A click on a button, a link or a pick box inside the table isn't a click on the row. See Events and actions.

Notation

```
["col",
  ["table", {"name": "produce", "value": [{"name": "Apples", "price": 2.4}, {"name":
"Honey", "price": 7.5}],
             "on": {"click": ["set!", "chosen", ["get", "$row", "name"]]}}],
  ["data", {"name": "chosen"}, ""],
  ["text", "Chosen: {{chosen}}"]]
```

Actions that change it

- Typed rows change with set!, as below. People change them by double-clicking in Edit.
- Saved records change with insert!, update!, delete! and clear!, and the table follows.
- A computed table can't be set; change what it reads.
- The picks change with the pick boxes and the **Clear** link. `set!` changes the rows, not the picks.

Expression

```
(set! team (concat team (list {name "Alan Turing" role "Analyst"})))
(map (get it "name") team)
⇒ (list "Ada Lovelace" "Grace Hopper" "Alan Turing")
```

On a phone and on paper

A table turns each row into a card when its own cell is narrow: under about 110 pixels per column, a little more with pick boxes and row buttons, and never wider than 560 pixels. On a phone that is most tables. A card has the first column as its title, the other columns as label and value, and the row buttons at its foot; empty values are left out. Sorting moves to a list in the toolbar, with *Original order* at the top. Group headings, totals, picking and search work as before.

In Page view and on paper every row shows: no search, no *Show more*, no pick boxes or row buttons, and groups are open. A long table runs on over as many pages as it needs.

Usual mistakes

- **"No rows yet".** The expression gave something that isn't a list of records: numbers or text are skipped. Wrap them in records: `(map {n it} xs)`.
- **A column shows empty.** The designer says *Not in the data, so they show empty: …* and offers **Remove them**. A field name in `columns` must match the records' exactly, with the

same capitals.

- **Searching for 1840 finds nothing.** The search looks at the text as shown, so it must be `$1,840.00` or part of it.
- **Showing 200 of 1000.** A table draws 200 rows at a time; **Show more** adds 200.
- **`"x" is not a table, so nothing in it can be picked`.** selected was given the name of another kind of cell. Name the table, and turn on **Picking rows**: with it off, nothing can be picked and the list is empty.
- **Picked rows jump after the data changes.** A row without an `id` is known by its place, so when the rows move the picks stay put. Give computed rows an `id` field.
- **`update! needs the id of a record`.** A row button called update! on a row that isn't a saved record. Row buttons that change data work on saved records: use **Saved records**, or a formula over `(rows "name")`.

These pages describe edgy as it is on `main`. .